

Chapitre 8 Tableaux statiques

Un tableau est une variable structurée et multivaluée, dont tous les éléments sont de même type. Ils sont utilisés dans les programmes pour travailler avec une grande quantité de données de type identique.

Les tableaux peuvent être classés en deux catégories : les tableaux unidimensionnels et les tableaux multidimensionnels.

Ce chapitre traite des tableaux statiques, dont la réservation et la libération mémoire est gérée de manière entièrement automatique et transparente pour le programmeur. C'est la raison pour laquelle on les appelle aussi parfois tableaux **automatiques**. Le chapitre 11 page 59 abordera les tableaux dynamiques, dont l'allocation et la libération de la mémoire doivent à l'inverse être entièrement et correctement gérés par le programmeur.

Déclaration des tableaux statiques

Un tableau statique est caractérisé par 3 éléments : son identificateur, son type et le nombre de ses éléments, appelé aussi dimension. En C++, la dimension d'un tableau est une **constante entière** qui précise le nombre d'éléments, le premier élément étant toujours indicé zéro.

Syntaxe

La déclaration d'un tableau statique se fait de la manière suivante :

```
type identificateur [dimension];           // unidimensionnel

ou

type identificateur [dim1] [dim2] [dim3]... // multidimensionnel
```

Remarques

- Un tableau statique se compose :
 1. d'un pointeur constant de nom `identificateur` et de type `type*`
 2. d'un emplacement de `dimension` éléments de type `type`, contigus en mémoire (c'est-à-dire rangés dans la mémoire les uns à la suite des autres, sans espace perdu entre chaque élément)

L'adresse de l'emplacement des `dimension` éléments est stockée dans le pointeur

Exemple

```
double pTab[8];
pTab est un pointeur de type double* qui contient l'adresse de l'emplacement des 8
double.
```

- On accède simplement à chaque élément du tableau à l'aide de l'opérateur `[]` et d'un indice entier.

Exemple

```
double pTab [8];
pTab[2] = 4;           // affecte 4 dans le 3ème élément du tableau
                        // attention, le 3ème élément porte le n° 2
                        // car le 1er élément porte le n° 0
cout << pTab[2] << endl; // affiche le 3ème élément du tableau
```

- L'indice du premier élément du tableau est toujours 0

Initialisation des tableaux

On peut initialiser un tableau statique en spécifiant sous la forme d'une liste un ensemble de valeurs séparées par des virgules et encadrées par les accolades de bloc {}.

Exemple

```
double pTab [8] = {2.1, 3.5, -4.2, 2.9, 1.5, 0.2, 4.3, 7.8};
```

Il est possible de ne pas donner de dimension lors de la déclaration d'un tableau initialisé par des valeurs lors de sa déclaration. Dans ce cas, la dimension du tableau est donnée par le nombre de valeurs qui servant à l'initialisation.

Exemple

```
double pTab [] = {2.1, 3.5, -4.2}; // équivalent à double pTab[3];
```

Lorsque la dimension est spécifiée, le nombre d'éléments de la liste d'initialisation doit être strictement égal à la dimension du tableau. C'est la raison pour laquelle on ne spécifie généralement pas le nombre d'éléments lorsqu'on déclare le tableau.

Représentation mémoire d'un tableau statique

Une représentation schématique de la mémoire occupée par un tableau statique aide à maîtriser son fonctionnement.

La déclaration d'un tableau `double pTab [8];` se présente en mémoire selon le schéma ci-dessous :

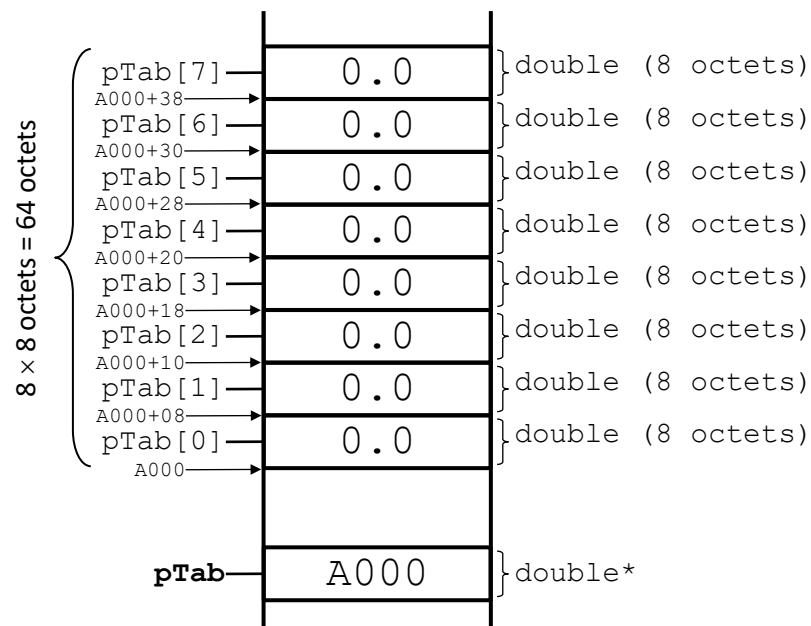


Figure 1 : représentation symbolique de la mémoire occupée par un tableau statique de réels.

Exemples d'utilisation de tableaux statiques

Déclaration

```
int pI[8]; // tableau de 8 entiers
char pC[80]; // tableau de 80 caractères
double pE[10]; // tableau externe de 10 réels
unsigned char pIma[512][512]; // tableau carré 512 x 512 (pixels)
```

Usage

Le nombre d'éléments d'un tableau statique étant connu, on utilise généralement une boucle `for` pour balayer l'ensemble de ses éléments (voir paragraphe « Choix des structures itératives » page 33).

Le premier élément des tableaux ayant l'indice 0, la boucle est généralement répétée de la valeur 0 à la valeur $N-1$, c'est-à-dire jusqu'à N non compris.

Exemple 1 : tableau monodimensionnel

```
const int nN = 10;
double pE [nN];
for (int i = 0 ; i < nN ; ++i)    // initialisation des éléments de pE
    pE[i] = (double) i;
```

Exemple 2 : tableau bidimensionnel

```
const int nN = 512;
const int nM = 512;
unsigned char pIma[nN][nM];

for (i = 0 ; i < nN ; ++i)        // initialisation de ima
    for (j = 0 ; j < nM ; ++j)
        pIma [i] [j] = (unsigned char) 0;
```

Relation entre tableaux et pointeurs

En C++, un tableau statique est un pointeur constant qui contient l'adresse du premier élément du tableau. Le pointeur est constant car l'adresse du premier élément du tableau ne doit pas être modifiée pour que la libération mémoire des éléments du tableau soit correctement réalisée. Le tableau suivant présente les équivalences d'écriture entre pointeurs et tableaux.

<code>pTab</code>	est équivalent à	<code>&pTab[0]</code>
<code>pTab+1</code>	est équivalent à	<code>&pTab[1]</code>
<code>pTab+i</code>	est équivalent à	<code>&pTab[i]</code>
<code>*pTab</code>	est équivalent à	<code>pTab[0]</code>
<code>*(pTab+1)</code>	est équivalent à	<code>pTab[1]</code>
<code>*(pTab+i)</code>	est équivalent à	<code>pTab[i]</code>
<code>*(pTab+i) = 1</code>	est équivalent à	<code>pTab[i] = 1</code>

Tableau 14 : relations entre tableaux et pointeurs.

Opérations sur les pointeurs

`pTab+i` est une adresse mémoire calculée par la formule `pTab + i · sizeof (*pTab)`.

Exemples

- Si `pTab` est un tableau de `double` qui contient l'adresse `A000`, `pTab + 3` est égal à `A000 + 3 · 8` octets, soit `A018`.
- Si `pTab` est un tableau de `short` qui contient l'adresse `B0D2`, `pTab + 7` est égal à `B0D2 + 7 · 2` octets, soit `B0E0`.

Tableaux de pointeurs

Il est possible de déclarer des tableaux de pointeurs en utilisant la syntaxe :

```
type* pNom [dimension];
```

Exemple

```
char* pTab1 [nN]; // pTab1 est un tableau de nN pointeurs sur caractère
                // à ne pas confondre avec
char (*pTab2) [nN]; // pTab2 est un pointeur sur un tableau de nN char
```

Chaînes de caractères

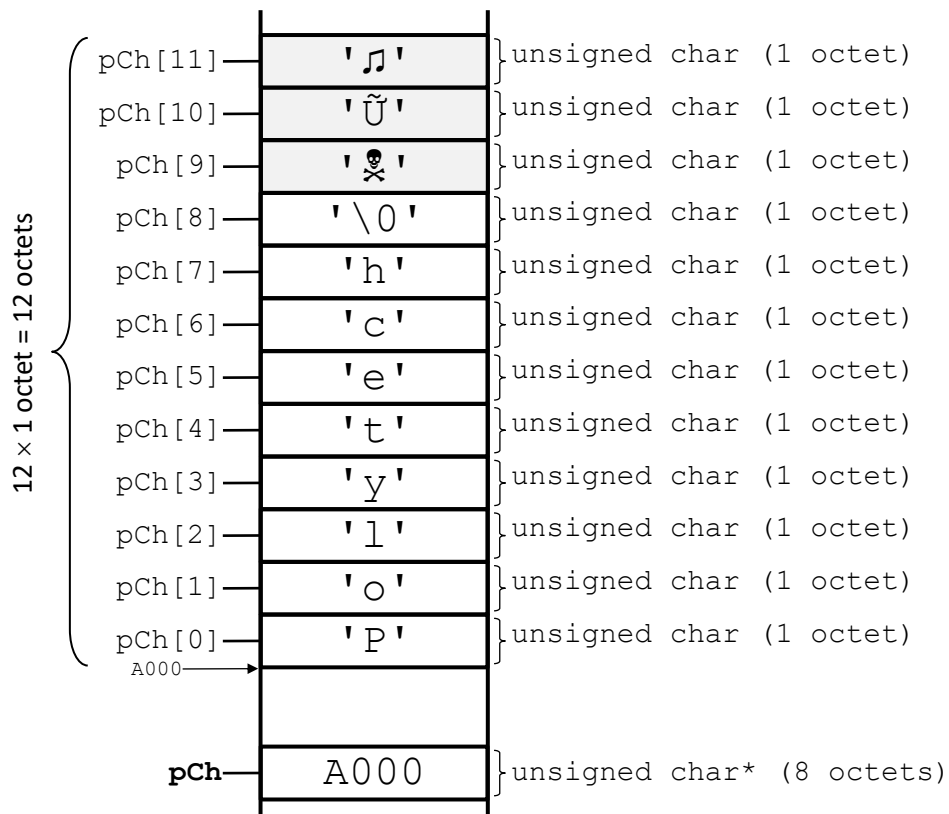
En C++, les chaînes de caractères sont définies comme des tableaux monodimensionnels de caractères non signés.

Syntaxe

```
unsigned char pCh [dimension]; // tableau unidimensionnel
```

Représentation mémoire des chaînes de caractères

La représentation de la chaîne de caractères `unsigned char pCh[12];` est par exemple :



Remarques

- Les chaînes de caractères stockées dans les tableaux peuvent être moins longues que le nombre total d'éléments du tableau. Pour identifier les caractères initialisés de ceux qui ne le sont pas, on utilise le caractère `'\0'`.
- Toute chaîne de caractères est supposée contenir le caractère `'\0'` qui indique la limite des caractères initialisés dans la chaîne. Il appartient donc aux programmeurs de gérer, directement ou en utilisant les fonctions appropriées, ce caractère.

- Pour stocker une chaîne de caractères de nN caractères, il faut prévoir de réserver un espace mémoire de $nN+1$ caractères afin de stocker le caractère `'\0'`.
- L'opérateur d'insertion dans les flux `<<` est le seul opérateur disponible du langage pour manipuler les chaînes de caractères. Par exemple :

```
unsigned char pCh [] = "Polytech Orléans";
cout << pCh << endl;
```

n'affiche pas le contenu du pointeur `pCh` (adresse du 1^{er} élément du tableau de caractères), mais tous les caractères du tableau (à partir de l'adresse contenue dans `pCh`) jusqu'au caractère `'\0'`.

Fonctions de manipulation des chaînes de caractères

Les fonctions de la bibliothèque `<string.h>` sont très utiles pour manipuler les chaînes de caractères. Voici quelques-unes des fonctions parmi le plus utilisées (les fonctions sécurisées sont indiquées par le suffixe `_s`):

- `size_t strlen (pSrc);` qui retourne le nombre de caractères initialisés de la chaîne `pSrc`, jusqu'au caractère `'\0'` non compris.
- `errno_t strcpy_s(char* pDest, size_t nSizeDest, const char* pSrc);` qui recopie la chaîne `pSrc` dans la chaîne `pDest` de dimension `nSizeDest`.
- `errno_t strcat_s(char* pDest, size_t nSizeDest, const char* pSrc);` qui ajoute les caractères de la chaîne `pSrc` à ceux de la chaîne `pDest` de dimension `nSizeDest`.
- `int strcmp(const char* pStr1, const char* pStr2);` compare les deux chaînes de caractères `pStr1` et `pStr2`. `strcmp()` retourne une valeur entière négative (< 0) si `pStr1` est alphabétiquement avant `pStr2`, nulle ($= 0$) si les deux chaînes `pStr1` et `pStr2` sont identiques, et une valeur positive (> 0) si `pStr1` est alphabétiquement après `pStr2`.
Attention si les opérateurs de comparaison ($<$ et $>$) peuvent être utilisés directement avec des chaînes de caractères, ils réalisent uniquement la comparaison des adresses des tableaux de caractères.
- `int atoi (const char* pSrc);`, `long ato (const char* pSrc);` et `double atof (const char* pSrc);` convertissent respectivement la chaîne de caractères `pSrc` en entier `int`, entier long `long` et réel `double`. En cas d'erreur, ces fonctions retournent la valeur 0, difficilement distinguable de la conversion de la chaîne de caractère "0". Pour contourner cette ambiguïté, il est possible d'utiliser la fonction `int sscanf (const char * pSrc, const char* pFormat, variables, ...);` qui convertit une chaîne de caractères en une série de valeurs.
- `int sprintf(char* pDest, const char* pFormat, variables, ...);` initialise la chaîne `pDest` avec les valeurs des variables passées en paramètres converties en caractères.



Pour les fonctions `strcpy_s()` et `strcat_s()`, il appartient au programmeur de vérifier que la taille de la chaîne de destination `pDest` comporte suffisamment de caractères pour recopier ou concaténer la chaîne source `pSrc`.

Toutes les fonctions de manipulation des chaînes de caractères supposent que le caractère de fin de chaîne `'\0'` est correctement positionné.

Initialisation des chaînes de caractères

Le cas le plus simple est celui des chaînes de caractères initialisées pendant leur déclaration. Cependant, des précautions doivent être prises car la manipulation des chaînes de caractères est bien souvent source d'erreur en C++.

Exemple 1

```
unsigned char pTexte[9] = "Polytech";
```

déclare une chaîne `pTexte` de 9 caractères, les 8 premiers prenant respectivement les valeurs 'P', 'o', 'l', 'y', 't', 'e', 'c', 'h' et le dernier la valeur '\0'. Pour éviter au programmeur de compter les caractères du texte, le compilateur calcule automatiquement la longueur de la chaîne (tableau de 9 caractères) si l'on écrit :

```
unsigned char pTexte[] = "Polytech";
```

Exemple 2

```
unsigned char pChaine[16];
pChaine = "Polytech";
```

entraîne une erreur car `pChaine` est un tableau statique, donc un pointeur constant, que l'affectation tente de modifier par l'adresse de la chaîne constante "Polytech".

Exemple 3

```
unsigned char* pChaine = NULL;
pChaine = (unsigned char*) "POLYTECH" ;
```

est correct car le pointeur `pChaine` peut être initialisé par l'adresse de la chaîne constante "Polytech".

Attention, toute tentative de modification de `pChaine` entraînera une erreur car il n'est pas possible de modifier la chaîne constante "Polytech" (`pChaine` pointe sur cette constante).

Vectorisation des tableaux statiques multidimensionnels

Les tableaux multidimensionnels sont vectorisés, c'est-à-dire que les éléments de chaque dimension sont rangés de manière contiguë en mémoire.

Il est ainsi possible d'utiliser indépendamment les deux écritures `pTab[i][j]` ou `pTab[i*nN+j]` pour accéder aux éléments d'un tableau `pTab[nM][nN]` à 2 dimensions :

```
int main (void)
{
    const int nM = 4;
    const int nN = 8;
    int pTab [nM][nN];

    for (int i = 0 ; i < nM ; ++i)
        for (int j = 0 ; j < nN ; ++j)
        {
            pTab[i][j] = 0;
        }

    for (int i = 0 ; i < nM ; ++i)           // écriture équivalente
        for (int j = 0 ; j < nN ; ++j)
        {
            pTab[i*nN+j] = 0;
        }
}
```